Axie Parts → Card Stats: Deterministic Algorithm V1

TL;DR

Every Axie NFT in your wallet generates **a unique, deterministic playable card** when you connect your Ronin Wallet. Same Axie + same algorithm version = always same card. No randomness. Audit-friendly.

This is the **core Web 2.5 hook**: 3M+ Axies in circulation = 3M+ unique cards, no manual art needed, mechanically distinct.

Inputs (read-only from on-chain + Axie GraphQL)

Field	Source	Example
`tokenId`	Ronin ERC-721 (Axie contract `0x32950db2a7164ae833121501c797d79e7b79d74c`)	`5234`
`class`	Axie GraphQL Gateway	`"Beast"`
`parts`	Axie GraphQL Gateway	`[{type:"Eyes",id:...]` (6 parts)
`birthDate`	Axie GraphQL Gateway	Unix timestamp
`level`	Axie GraphQL Gateway (in-game level if exists)	`15`

Outputs (server-signed card spec)

```ts
interface AxieCardStats {
  cardId: string;           // unique: `axie-${tokenId}`
  name: string;             // "{class} Axie #{tokenId}"
  classType: AxieClass;     // 9 classes
  level: number;            // 1-8 (capped)
  atk: number;              // 800-2800
  def: number;              // 600-2400
  burns: number;            // 0-2 (sacrifice cost)
  effect?: CardEffect;      // 0-1 effects per card
  rarity: 'Common' | 'Rare' | 'Epic' | 'Legendary';
  signedAt: number;         // server timestamp
  signature: string;        // HMAC server-signed
}
```

Algorithm V1 — Deterministic mapping

Step 1: Class assignment

```
classType = axie.class  // direct copy from GraphQL
```

Step 2: Base stats from class

```
baseAtk = CLASS_BASE_ATK[classType]   // see table below
baseDef = CLASS_BASE_DEF[classType]
```

Class	Base ATK	Base DEF	Bias
Beast	1700	1200	offensive
Bug	1500	1100	offensive-utility
Mech	1900	1400	offensive
Plant	1100	1900	defensive
Reptile	1400	1700	tanky
Dusk	1300	1500	balanced

Aqua	1500	1500	balanced
Bird	1800	900	glass cannon
Dawn	1600	1300	utility

Step 3: Parts modifier sum

For each of the 6 parts (eyes, ears, mouth, horn, back, tail), apply a modifier from the lookup table. Same-class parts give bigger modifiers.

```
for each part in axie.parts:
  modifier = PARTS_MODIFIER_TABLE[part.id]
  if part.class === axie.class:
    atk += modifier.atk * 1.2  // synergy bonus
    def += modifier.def * 1.2
  else:
    atk += modifier.atk
    def += modifier.def
```

Step 4: Cap normalization

```
atk = clamp(atk, 800, 2800)
def = clamp(def, 600, 2400)
```

Step 5: Level + burns

```
// Level proxy: birthdate antiquity (older = more "earned" = higher
level)
// + part rarity tier (mystic parts → higher level cap)
ageMonths = (now - axie.birthDate) / (30 * 24 * 3600 * 1000)
rarityTier = max(part.rarityTier for part in axie.parts)  // 1-3

level = clamp(round(ageMonths / 6 + rarityTier), 1, 8)

// Burns required to deploy:
// L1-4 → 0 burns, L5-6 → 1 burn, L7-8 → 2 burns
burns = level <= 4 ? 0 : level <= 6 ? 1 : 2
```

Step 6: Effect mapping (THE MAGIC)

Each part has a possible effect. The **horn** part determines the primary effect (most distinctive). If horn is unmapped, fall back to mouth, then back.

Part ID (example)	Effect	Description
`horn.lagging`	`onAttack: 30% chance draw 1`	Draw extra cards on attack
`horn.shoebill`	`aura: +200 ATK to all your Bug Axies`	Buff aura
`horn.imp`	`onSummon: deal 200 damage to opp LP`	Burn on entry
`mouth.lips`	`aura: +200 ATK to self`	Self-buff
`mouth.tiny-turtle`	`onAttack: heal 100 LP`	Lifesteal
`back.snail-shell`	`passive: +500 DEF in DEF position`	Defensive buff
`back.hermit`	`onDefend: reflect 30% damage`	Thorns
`tail.furball`	`onDeath: opp loses 300 LP`	Death rattle
`tail.cottontail`	`onDeath: draw 1 card`	Cycling
`eyes.zigzag`	`onAttack: 20% pierce DEF`	Anti-tank
`eyes.gas`	`passive: +10% ATK if no other axie on field`	Solo bonus
`ears.lotus`	`aura: heal 50 LP per turn`	Sustain
`ears.mint`	`passive: immune to traps`	Anti-counter

V1 lookup table covers ~30-40 of the most common parts. Full coverage of ~150 parts in V2.

Fallback: if no part is mapped, card has **no effect** (vanilla stats only). Still playable, just less distinctive.

Step 7: Rarity classification

Based on number of mapped effects + same-class part count:

```
synergyParts = count(part.class === axie.class for part in parts)
mappedEffects = count(part is in effect lookup)

if synergyParts >= 5 && mappedEffects >= 4: rarity = 'Legendary'
elif synergyParts >= 3 && mappedEffects >= 3: rarity = 'Epic'
elif synergyParts >= 1 && mappedEffects >= 1: rarity = 'Rare'
else: rarity = 'Common'
```

Step 8: Server signature

```
signature = HMAC_SHA256(
  secret: server.SIGNING_KEY,
  payload: { cardId, atk, def, level, effect, signedAt }
)
```

The signature is verified by Colyseus on match start. **Cliente nunca puede modificar stats**.

Worked example

Input: Axie #5234, Beast class. **Parts**:

- eyes: `puppy` (Beast)
- ears: `pup` (Beast)
- mouth: `axie-kiss` (Beast)
- horn: `little-branch` (Plant) → cross-class
- back: `furball` (Beast)
- tail: `cottontail` (Beast)

Calculation:

```
classType = 'Beast'
baseAtk = 1700, baseDef = 1200

parts modifiers:
  eyes.puppy (Beast, synergy): +50 atk * 1.2 = +60, +30 def * 1.2 = +36
  ears.pup (Beast, synergy): +30 atk * 1.2 = +36
  mouth.axie-kiss (Beast, synergy): +80 atk * 1.2 = +96
  horn.little-branch (Plant, cross-class): +40 atk, +60 def
  back.furball (Beast, synergy): +50 def * 1.2 = +60
  tail.cottontail (Beast, synergy): +20 atk * 1.2 = +24, +20 def * 1.2
= +24

atk = 1700 + 60 + 36 + 96 + 40 + 24 = 1956
def = 1200 + 36 + 60 + 60 + 24 = 1380

level: ageMonths=18 / 6 = 3 + rarityTier=1 = 4
burns = 0 (L≤4)

effect: horn.little-branch is unmapped → fallback to mouth.axie-kiss
(mapped: aura +200 ATK to self)

synergyParts = 5/6 (all but horn) → high
mappedEffects = 1 (only mouth) → low
→ rarity = 'Rare'
```

Output card:

```
Name: "Beast Axie #5234"
Class: Beast
Level: 4
ATK: 1956 / DEF: 1380
Burns: 0
Effect: "Aura — +200 ATK to self"
Rarity: Rare
```

Audit notes

Why this is fair

- **Deterministic**: same Axie + same algorithm version → always same card. No hidden randomness.
- **Server-signed**: stats can't be modified by client.
- **Public algorithm**: this document IS the spec. Sky Mavis can audit it.
- **Versioned**: when we update the algorithm (V2 with more parts), old card NFTs (if minted) keep V1 stats — no rug pull.

Why this maintains F2P balance

- **Stat ranges overlap**: starter axie deck (1500-2200 ATK / 1000-1900 DEF) vs NFT axies (1500-2500 / 1000-2200). NFT axies are not categorically stronger.
- **Effects are side-grades**: an NFT card with "draw 1 on attack" isn't strictly better than a starter with no effect. Skill in deck-building decides.
- **No "must-have" NFT cards**: the ladder meta is reachable with any combination of starter + earned cards. Validated by tournament data post-launch.

Why this is sustainable for Sky Mavis

- **Demand for rare parts** (mythic-tier eyes/horns/etc) creates secondary market pressure on Axie Marketplace.
- **No new minting required**: existing 3M+ Axies become 3M+ cards day 1.
- **Open algorithm** = competitor games can't easily replicate the deterministic mapping without us — provides moat.

Implementation reference

- **Library**: `apps/web/src/lib/axie-card-algorithm.ts` (V1)
- **Tests**: `apps/web/src/lib/axie-card-algorithm.test.ts`
- **GraphQL client**: `apps/web/src/lib/axie-graphql-client.ts`
- **Demo page**: `/my-axies` (live at https://axie-duel.vercel.app/my-axies)

V2 roadmap

- Cover all ~150 known parts in lookup table
- Class triangle effect synergies (e.g., Beast horn on a Plant body has different effect than on a Beast body)

- Mystic parts (1-of-a-kind Axies) get unique legendary effects
- Birthdate-based "Origins veteran" badge (collectible, no power impact)

Questions for Sky Mavis: we're open to refining this algorithm based on game design philosophy of the ecosystem. Happy to align with internal balancing standards.